

Helix CTF

How each fragment was solved

look where you're not supposed to · notice the tell · exploit it · submit the six-character code

F1 · Login

Bug class: client-side recon

The login form needs three things. The app leaks one in each of three places.

Field	Where it lives	How to find it
Username	HTML comment near the form	DevTools → Elements
Password	console.log with no [prefix]	DevTools → Console
Access code	sessionSeed in /api/config	DevTools → Network

Takeaway. Anything in the client is public. Markup, console, and config are all readable.

F2 · Privilege escalation

Bug class: server trusts client-supplied role

A plain POST /api/profile (as viewer) is already a gift: the response leaks availableRoles: [..., "admin"] and a free accessToken. Replay it with the leaked header and an admin claim:

```
POST /api/profile
X-Access-Token: xK9mQ2vL8nR3
Content-Type: application/json
{ "role": "admin" }
```

The server obliges.

Takeaway. Authorization belongs on the server. Never let the client pick its own role.

F3 · Cookie tampering

Bug class: client-side state controls server access

After login a `clearance=basic` cookie is set. On `/projects`, "Project Gamma" is locked.

- 1 DevTools → Application → Cookies
- 2 Change `clearance` from `basic` to `admin`
- 3 Reload `/projects` — Gamma unlocks.

Takeaway. *Anything a client can mutate, a client will mutate. Cookies are not credentials.*

F4 · Hidden in plain sight

Bug class: off-screen is not unrendered

The `/reports` narrative hints at "what the eye does not see." DevTools → Elements turns one up:

```
<div data-score-code="ED7015"
style="left: -9999px; opacity: 0">...</div>
```

The code sits in an attribute on a node positioned off-screen.

Takeaway. *Rendering happens regardless of CSS. If it is in the DOM, it is in the response.*

F5 · Obfuscated JS resolve

Bug class: reverse engineering

`/analytics-helper.js` looks like noise: hex variable names, char-code arrays, decoy `_trackEvent` dead code. Past all of it, one function is attached to `window`:

```
window._resolve(token, seed)
```

`token` is the `accessToken` from F2. `seed` is the `sessionSeed` from F1. Call it from the console.

Takeaway. *Obfuscation buys time, not safety.*

F6**F6 · JWT forge**

Bug class: `alg:none signature bypass`

The `/audit-log` page asks for a Bearer JWT and says "Auditor role required." The input placeholder is `eyJhbGciOiJIub25lI...`, which is base64 for `{"alg":"none"...`. The page is telling you the attack.

Forge a header.payload. pair with an empty signature:

```
const b64url = o =>
  btoa(JSON.stringify(o))
  .replace(/=+$/, '').replace(/\+/g, '-').replace(/\//g, '_');
const token =
  b64url({ alg: 'none', typ: 'JWT' }) + '.' +
  b64url({ role: 'auditor' }) + '.';
```

Paste, click Load Entries.

Takeaway. Always verify the signature. `alg:none` should never be accepted server-side.

F7**F7 · IDOR**

Bug class: `predictable identifiers`

GET `/api/reports` returns 47 reports, and one extra field:

```
{ "reports": [ ... ], "nextPageStart": 9999 }
```

That number is the confession. Visit `/api/reports/9999` and the server hands back an internal post-mortem with the score code embedded.

Takeaway. Sequential IDs are confessions. Use opaque tokens, and authorize on the object, not on the URL pattern.

Every fragment is something the app told the player itself. The skill is not breaking in. It is noticing what was already open.